

Appendix A – The Practical Cheatsheet

This section is designed to be printable and standalone. It compresses the rest of the whitepaper into the decisions a team will face.

The minimal SKILL.md

Copy-paste this. Adjust the placeholders. Done.

```
---
name: skill-name
description: |
  [What it does in one verb-led sentence.] Use this skill when the user
  [trigger phrase 1], [trigger phrase 2], or [trigger phrase 3].
  Do NOT use for [anti-trigger 1] or [anti-trigger 2].
version: 1.0.0
license: MIT
allowed-tools: [Optional] Read Bash Write
metadata:
  author: [Optional] your-handle
---

# Skill Name

## When to use
- [Concrete scenario]
- [Concrete scenario]

## When NOT to use
- [Out-of-scope scenario]

## Workflow
1. [Step]
2. [Step]
3. See `references/advanced.md` for [edge case].
```

Continues next page...

```
## Examples
- Input: "... " → Output: "... "

## Output format
- Use `assets/template.md` etc.

## Anti-patterns to avoid
- Don't [...]
```

The folder structure.

```
skill_name/
├── SKILL.md           # Required: YAML frontmatter + markdown instructions
├── scripts/          # Optional: executable helper scripts (Python, Bash)
├── references/        # Optional: supplementary context loaded as needed
├── assets/           # Optional: files used in output (templates, resources)
└── ...               # Any additional files or directories
```

Naming

- Directory name: snake_case
- Skill name: kebab-case
- Prefer gerund form: `processing-pdfs`, not `pdf-processor`
- Avoid generic names: `helper`, `utils`, `tools`, `data`
- Avoid vendor prefixes: `claude-*`, `gemini-*`, `anthropic-*`
- Avoid internal jargon outsiders won't recognize

The description field

The description is the routing algorithm. Spend more time here than anywhere else.

- State **what it does** AND **when to use it**
- Front-load trigger keywords ("Generate a commit message...", not "This skill helps with...")
- Include **when NOT to use** to prevent over-triggering
- Be pushy when the model under-triggers
- ≤200 chars for API; ≤1024 chars in YAML. Most authors aim for ~50 words

The five rules

1. **One skill, one job.** If you cannot describe what the skill does in one sentence, it is two skills. Decompose before writing.
2. **Descriptions are an interface.** The agent picks skills by reading descriptions. A vague description means an unused skill.
3. **Skills are dependencies.** Treat them like libraries. Version them, pin them, review them in PRs. A skill without a test is a hope, not a capability.
4. **The right team owns the right skill.** Domain experts own domain skills. Do not let the AI team become a bottleneck for the organization's domain knowledge.
5. **The agent runtime is interchangeable.** Do not tie skills to one runtime. Portability is part of the value.

The quality principles

1. **Run the task yourself first.** Real failure produces signal. Speculation produces noise.
2. **Give the reason, not just the rule.** Models generalize wonderfully to edge cases when they understand *why* an instruction exists. If you find yourself typing "ALWAYS" or "NEVER" in caps, pause and try explaining the rationale instead.
3. **Every line should earn its place.** Keep gotchas, exact commands, business logic, anti-patterns. Cut boilerplate the model already knows such as "always validate output".
4. **One skill, one job.** If the description needs "and" between unrelated capabilities, split it.
5. **Make instructions verifiable.** If the agent can't tell whether it followed the rule, the rule is too vague.
6. **Bundle what repeats.** Helper code the agent keeps re-deriving belongs in `scripts/`.

Do's and Don'ts

Do:

- Start small and concrete
- Spend disproportionate time on the description field. It is the routing algorithm. Always include what it does, when to use, and when not to use
- Trust progressive disclosure
- Bundle deterministic work in `scripts/`, not as instructions
- Treat Skills as code

- Remember: Skills + MCPs, not Skills vs. MCPs
- Have test cases and controlled, programmatic evaluation (see **Section 4** for how to actually do this)

Don't:

- Write vague descriptions like "helps with documents". Use specific verbs, trigger phrases, and a when-not-to-use clause
- Write SKILL.md bodies over 5,000 words. Move detail to [references/](#)
- Hard-code paths or secrets
- Embed "always do X" rules that are more appropriate for AGENTS.md
- Install untrusted third-party libraries or Skills without scanning
- Reinvent MCP as scripts

Skill smells (revise if you see these)

- **Over 5,000 words.** Probably two skills, or reference material that should live in [references/](#)
- **Two domain teams could plausibly own it.** Not yet decomposed. Split along team boundaries
- **You can't write three test cases for it.** Description is too vague; skill does too many things
- **It does not reference any other resource.** Might just be a long instruction that belongs in the system prompt

- **You keep adding "edge cases" sections.** Each edge case probably wants its own skill
- **Its description starts with "a helpful skill for..."**. Rewrite. The description should name the trigger, the inputs, and the output

Eval coverage checklist

A skill is "evaluated" only when all four are satisfied:

- ☐ **Trigger.** Positive AND negative test cases. Target 90% trigger accuracy
- ☐ **Execution.** Correct outputs across a representative range of inputs
- ☐ **Regression.** Adding this skill causes zero drops in the existing library suite
- ☐ **Token budget.** Co-loaded with 5 to 15 frequently-active skills, does not degrade unrelated turns

Any failure holds the skill at the draft tier, regardless of happy-path performance.

Deployment checklist

- ☐ Frontmatter validates (lint passes)
- ☐ Description includes what + when + when-not
- ☐ Scripts have unit tests passing in CI
- ☐ Eval suite passes in CI with min-pass threshold
- ☐ Security scan clean (no secrets, no untrusted deps)

- ☐ Description reviewed by someone other than the author
- ☐ Cross-tool install paths tested if shipping publicly
- ☐ Org-level admin provisioning updated (if applicable)

One-line mental model

System prompt = instinct. AGENTS.md = project README. Tools / MCP = hands. RAG = library. Skills = the runbook the experienced colleague hands you on day one, and that the AI never forgets.

Where to start tomorrow

1. Take your most experienced practitioner aside for an hour. Ask them to narrate three workflows they do regularly. Record it.
2. Pick the most repeated workflow. Run the prompts yourself, without any skill loaded. Note where the agent fails. Or let an agent similar to ()
3. Draft a SKILL.md from the transcript. Write three eval cases (two positive, one negative) before drafting the body.
4. Ship to a read-only tier. Test in production-like conditions. Iterate the description until trigger accuracy clears 90%.

Repeat. Build the library one workflow at a time. Resist the urge to let an LLM generate fifty skills on day one.